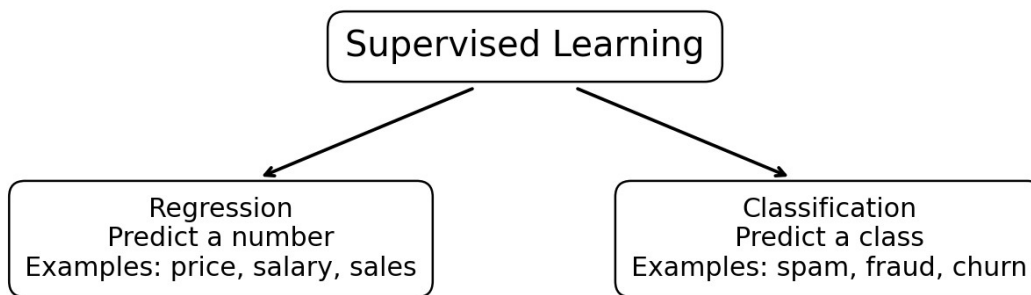


MODULE 11

SUPERVISED LEARNING

Detailed Student Study Material

*Concept explanations, model workflows, evaluation guidance, and practical question-answer exercises
with dummy data*



Prepared from the Module 11: Supervised Learning syllabus outline

How to Use This Material

This tutorial expands the Module 11 syllabus into a student-friendly reference. Each section explains the concept, shows the workflow, highlights important terms, and connects the topic to practical machine learning tasks. The final section contains hands-on questions with complete Python answers using self-contained dummy datasets.

- Read the concept sections first and identify the target variable in each example.
- For every problem, decide whether it is regression or classification before choosing an algorithm.
- Run the practical code line by line in Jupyter Notebook, Google Colab, VS Code, or any Python environment.
- Change the dummy values and observe how predictions and metrics change.
- Use the review questions to prepare for theory, viva, and practical examinations.

Learning Outcomes

- Explain supervised learning and the role of labeled data.
- Differentiate regression and classification problems.
- Build Linear Regression, Logistic Regression, KNN, Naive Bayes, Decision Tree, Random Forest, and boosting models.
- Understand train-test splitting, model fitting, prediction, and evaluation.
- Compare multiple models using suitable metrics instead of relying only on accuracy.
- Interpret feature importance and connect model findings to business decisions.
- Use AI tools as an assistant for model comparison, feature analysis, and experiment planning while validating recommendations.

1. Supervised Learning Fundamentals

Supervised learning is a machine learning approach in which an algorithm learns from examples that contain both input variables and known outputs. The known output acts as a teaching signal. The model learns a mapping from inputs to outputs and then uses that learned relationship to predict the output for new, unseen inputs.

Key idea: Supervised learning = input features + known labels during training -> learned model -> prediction on unseen data.

1.1 What Is Supervised Learning?

Suppose a company has historical employee records containing years of experience and salary. If the goal is to predict salary for a new employee, the historical salary is the label and experience is a feature. During training, the model sees many feature-label pairs and tries to learn a mathematical relationship that minimizes prediction error.

Experience (years)	Salary
1	25,000

3	40,000
5	62,000
8	85,000

In mathematical form, supervised learning tries to learn a function f such that $f(X)$ is close to Y . Here X represents one or more features and Y represents the target or label.

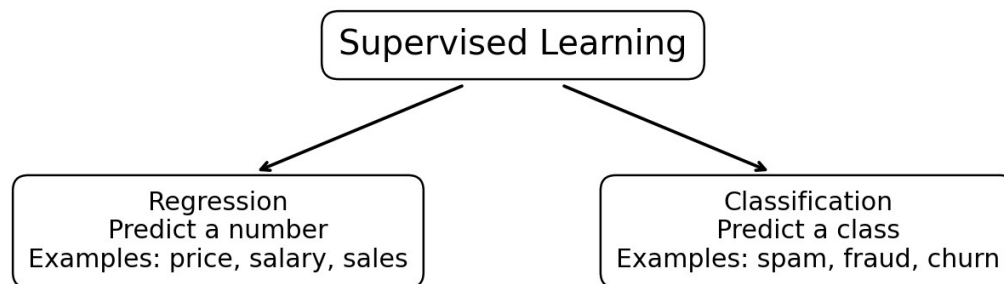
1.2 Learning from Labeled Data

A labeled dataset contains observations for which the correct target value is already known. Labels may be numeric, such as a house price, or categorical, such as spam/not spam. The reliability of a supervised model depends strongly on label quality. Incorrect, inconsistent, or biased labels can cause the model to learn the wrong pattern.

Email text	Label
"Win a free prize now"	Spam
"Team meeting at 10 AM"	Not Spam
"Exclusive offer - click here"	Spam

Important: A large dataset is not automatically a good dataset. Incorrect labels, duplicated records, leakage, and unrepresentative samples can damage model quality.

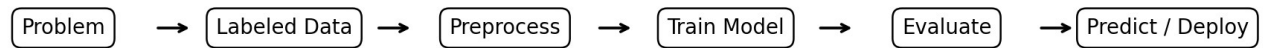
1.3 Classification vs Regression



Aspect	Regression	Classification
Target type	Continuous numeric value	Category / class
Examples	price, salary, temperature, demand	spam, fraud, churn, disease
Typical metrics	MAE, MSE, RMSE, R2	accuracy, precision, recall, F1
Typical algorithms	Linear Regression, Random Forest Regressor, XGBoost Regressor	Logistic Regression, KNN, Naive Bayes, Decision Tree, Random Forest

Common exam trap: Logistic Regression contains the word "Regression" but is mainly used as a classification algorithm.

1.4 Model Training Workflow



1. Define the problem and identify the target variable.
2. Collect and understand labeled data.
3. Clean the data and handle missing or inconsistent values.
4. Separate features (X) and target (y).
5. Split the data into training and testing sets.
6. Choose an appropriate algorithm.
7. Train the model using the training data.
8. Generate predictions for unseen test data.
9. Evaluate the model using suitable metrics.
10. Tune, compare, document, and deploy the final model.

1.5 Practical Use Cases

Domain	Example supervised learning task	Problem type
Banking	Predict loan default	Classification
Healthcare	Predict treatment cost	Regression
E-commerce	Predict customer churn	Classification
Real estate	Predict house price	Regression
Manufacturing	Detect defective product	Classification
Logistics	Estimate delivery time	Regression

2. Regression Algorithms

Regression is used when the target variable is a continuous numeric quantity. A regression model attempts to estimate how the target changes when one or more input features change.

2.1 Linear Regression

Linear Regression models the relationship between a target and one or more features using a straight-line relationship. In simple linear regression with one feature, the equation is:

$$y = mx + c$$

Here y is the predicted target, x is the input feature, m is the slope, and c is the intercept. The algorithm selects values for m and c that make the predicted values as close as possible to the actual values, typically by minimizing squared error.

Experience	Actual Salary	Possible Predicted Salary
1	25,000	26,000
3	40,000	41,000
5	62,000	60,000

2.2 Multiple Linear Regression

Multiple Linear Regression extends the same idea to several features. For a house-price model, area, bedrooms, property age, and distance from the city may all be used together.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Interpretation: A coefficient describes how the predicted target changes with a feature when the other features are held constant. Coefficients must be interpreted carefully when features are highly correlated or use very different scales.

2.3 Regression Assumptions

Assumption	Meaning	Why it matters
Linearity	The relationship is reasonably linear	Strong non-linearity can lead to systematic errors
Independent errors	Residuals are not strongly dependent	Violation may make statistical conclusions unreliable
Homoscedasticity	Residual spread is approximately constant	Changing variance can affect inference and error interpretation
Residual normality	Residuals are roughly normal for classical inference	Important mainly for confidence intervals and tests
Low multicollinearity	Predictors are not excessively redundant	High correlation can destabilize coefficients

A residual is the difference between an actual value and a predicted value: $\text{residual} = \text{actual} - \text{predicted}$. Residual analysis helps identify whether the model is missing patterns.

2.4 Regression Prediction Workflow

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

data = pd.DataFrame({
    "experience": [1,2,3,4,5,6,7,8],
    "salary": [25000,30000,37000,45000,52000,61000,70000,80000]
})

X = data[["experience"]]
y = data["salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

2.5 Regression Applications and Metrics

Metric	Meaning	Interpretation
MAE	Mean absolute error	Average absolute prediction error; easy to explain in target units
MSE	Mean squared error	Penalizes large errors more strongly
RMSE	Square root of MSE	Error measured in the same unit as the target
R2	Coefficient of determination	How much target variation is explained by the model; higher is generally better

Metric selection: No metric should be interpreted alone. A model can have a good R2 but still make large errors for specific important cases.

3. Classification Algorithms

Classification is used when the target is a class or category. The model learns patterns that separate observations into classes. Some classifiers produce only a class, while others also estimate class probabilities.

3.1 Logistic Regression

Logistic Regression estimates the probability of a class, especially in binary classification. A linear combination of features is passed through the sigmoid function, which converts the result into a value between 0 and 1. A threshold such as 0.5 is then used to convert the probability into a class label.

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

Predicted probability	Threshold	Predicted class
0.82	0.50	Class 1
0.31	0.50	Class 0

3.2 K-Nearest Neighbors (KNN)

KNN is a distance-based algorithm. To classify a new observation, it finds the K closest training points and assigns the class that appears most often among those neighbors. In regression mode, it can average neighboring target values.

- Small K can make the model sensitive to noise.
- Large K produces smoother decisions but may ignore local structure.
- Feature scaling is important because distance is affected by feature magnitude.
- Prediction can be slow for very large training datasets because neighbors must be searched.

3.3 Naive Bayes

Naive Bayes is a probabilistic family of classifiers based on Bayes theorem. The "naive" assumption is that features are conditionally independent given the class. Although this assumption is often not fully true, Naive Bayes can work very well for text classification and other high-dimensional tasks.

Variant	Typical data
Gaussian Naive Bayes	Continuous numeric features
Multinomial Naive Bayes	Word counts / frequencies
Bernoulli Naive Bayes	Binary features such as word present/not present

3.4 Classification Workflow

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

3.5 Classification Metrics

Metric	Question it answers
Accuracy	What fraction of all predictions were correct?
Precision	When the model predicts positive, how often is it correct?
Recall	Of all actual positives, how many did the model detect?
F1-score	What is the harmonic balance between precision and recall?
Confusion matrix	How many true positives, true negatives, false positives, and false negatives occurred?

Imbalanced data: A model that predicts the majority class for every record can achieve high accuracy while failing completely on the minority class. Use precision, recall, F1, ROC-AUC/PR-AUC, and business cost when appropriate.

4. Tree-Based Models

4.1 Decision Trees

A Decision Tree repeatedly divides the dataset using feature-based rules. Internal nodes represent questions, branches represent outcomes of those questions, and leaf nodes contain the final prediction. Trees can handle nonlinear relationships and feature interactions without requiring feature scaling.

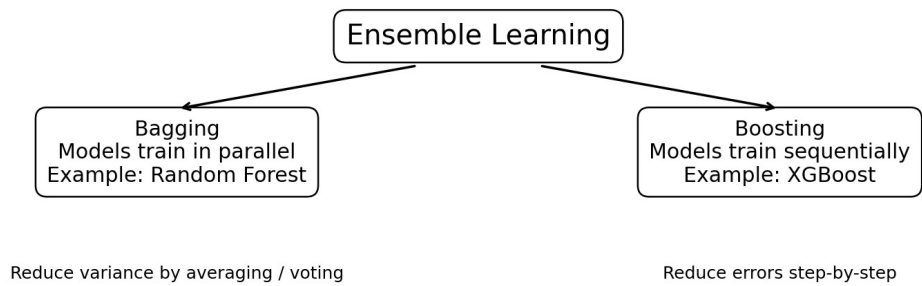
Tree component	Meaning
Root node	First split
Internal node	Intermediate decision
Branch	Result of a split
Leaf node	Final class or numeric prediction

- Classification trees commonly use Gini impurity or entropy to choose splits.
- Regression trees commonly minimize squared error or related variance criteria.
- Deep unrestricted trees can overfit. Common controls include `max_depth`, `min_samples_split`, and `min_samples_leaf`.

4.2 Random Forest

Random Forest is an ensemble of many decision trees. Each tree is trained on a bootstrap sample of the training data and considers a random subset of features at each split. Classification combines votes; regression averages numeric predictions. This reduces the instability of a single tree and often improves generalization.

4.3 Ensemble Learning Concepts



Method	Training style	Main idea	Example
Bagging	Parallel	Average many high-variance models	Random Forest
Boosting	Sequential	Each model corrects earlier errors	XGBoost

4.4 Feature Importance

Tree ensembles can provide feature importance scores that summarize how much features contribute to reducing prediction error or impurity. These scores are useful for exploration but should not be treated as proof of causation. Correlated variables can share or distort importance.

Feature	Illustrative importance
Income	0.40
Credit score	0.35
Debt ratio	0.17
Age	0.08

4.5 Tree-Based Decision Making

Example rule learned by a tree:

```
IF credit_score > 730
  AND debt_ratio < 0.35
  AND income > 60000
THEN predicted_risk = "Low"
ELSE continue to another branch
```

This rule-like structure makes Decision Trees easy to explain. Random Forests are more complex because many trees are combined, but feature importance and local explanation tools can still provide useful interpretations.

5. Boosting Concepts and XGBoost

5.1 What Is Boosting?

Boosting builds a strong predictive model by combining a sequence of weaker models. Each stage focuses on errors or residuals left by previous stages. Rather than training all models independently, boosting learns progressively.

Key idea: Bagging reduces variance through parallel diversity; boosting reduces errors by sequential correction.

5.2 XGBoost Fundamentals

XGBoost stands for Extreme Gradient Boosting. It is a high-performance implementation of gradient-boosted decision trees and is widely used for structured/tabular data. It supports both classification and regression, includes regularization, and provides many controls for learning rate, tree depth, row sampling, and feature sampling.

Hyperparameter	Purpose
n_estimators	Number of boosting trees
learning_rate	Contribution of each new tree
max_depth	Maximum tree depth
subsample	Fraction of training rows used per tree
colsample_bytree	Fraction of features used per tree

5.3 Model Improvement Concepts

- Improve data quality and labels.
- Engineer meaningful features.
- Remove leakage and irrelevant variables.
- Use cross-validation for reliable model comparison.
- Tune hyperparameters systematically.
- Handle class imbalance where needed.
- Regularize or simplify models to reduce overfitting.

5.4 Performance Optimization

Performance optimization should balance predictive quality, training cost, inference speed, model complexity, and interpretability. A deeper model can fit more complex relationships but may overfit. A smaller learning rate often needs more trees. Subsampling can reduce overfitting and training cost. The correct settings depend on validation results, not on a universal formula.

5.5 Practical Uses of Boosting

Application	Possible target
Fraud detection	Fraud / genuine
Customer churn	Churn / stay

Credit scoring	Default probability
Sales forecasting	Future sales amount
Insurance risk	Claim cost / claim class
Manufacturing	Defect probability

6. Model Training Workflows

6.1 Data Splitting

Training and testing on the same data creates an unrealistically optimistic evaluation. A standard workflow divides data into training and testing sets. Some projects also use a validation set for tuning. Cross-validation is another common way to estimate how well a model generalizes.

Split	Purpose
Training set	Learn model parameters
Validation set	Choose hyperparameters and compare variants
Test set	Final unbiased evaluation after model selection

Data leakage: Never let information from the test set influence training or preprocessing decisions. Fit scalers, imputers, encoders, and feature selectors only on training data and then transform validation/test data using the fitted objects.

6.2 Training Models: Underfitting and Overfitting

Situation	Training performance	Test performance	Interpretation
Underfitting	Poor	Poor	Model is too simple or features are insufficient
Good fit	Good	Good	Model generalizes reasonably
Overfitting	Very high	Much lower	Model memorizes training-specific patterns

6.3 Generating Predictions

After a model is trained, `.predict()` returns predicted labels or values. Many classifiers also provide `.predict_proba()`, which is useful when thresholds or ranking decisions matter. Production inputs must receive the same preprocessing used during training.

6.4 Comparing Models

Model comparison should include both statistical performance and practical constraints. The most accurate model is not always the best operational choice.

Factor	Why it matters
Predictive metric	Measures task performance
Latency	Important for real-time systems
Memory/model size	Important on mobile/edge systems
Explainability	Important in regulated/high-impact use cases
Training time	Affects iteration speed and cost
Stability	Model should perform consistently across data splits

6.5 Experiment Tracking

Experiment tracking records what was tried and what happened. A useful experiment record includes dataset version, preprocessing, feature set, algorithm, hyperparameters, random seed, metrics, run date, and saved model version. Tools such as MLflow or Weights & Biases can automate this process.

Run	Model	Key setting	Metric
Exp-01	Logistic Regression	C=1.0	F1=0.82
Exp-02	Random Forest	200 trees	F1=0.87
Exp-03	XGBoost	lr=0.05	F1=0.89

7. Feature Importance and Interpretability

7.1 Important vs Irrelevant Features

A useful feature contains information related to the target. An irrelevant feature contributes little or only adds noise. IDs, random codes, future information, and duplicated variables often require special attention. Feature selection can reduce complexity, improve interpretability, and sometimes improve generalization.

7.2 Feature Impact

Feature importance says how much a feature matters to model decisions, but it may not show direction. For example, a churn model may use "days since last login" strongly, and further analysis may show that larger values increase churn probability. Techniques such as coefficients, permutation importance, partial dependence, and SHAP can help explain direction and local impact.

7.3 Model Interpretability

Technique	Useful for
Linear/logistic coefficients	Direction and magnitude in simple linear models
Tree visualization	Rules learned by a Decision Tree
Permutation importance	Model-agnostic global feature importance
SHAP values	Global and per-prediction explanations

Causation warning: A feature being important to a predictive model does not prove that changing that feature will cause the target to change.

7.4 Business Insights from Models

A predictive model is most useful when its output supports an action. If a churn model identifies low engagement, repeated support complaints, and long inactivity as strong risk indicators, a business may prioritize retention outreach. Model results should therefore be translated into clear, testable, and ethical business decisions.

8. Thinking with AI

AI assistants can accelerate learning and experimentation, but generated recommendations must be checked against the actual dataset, documentation, and measured results. AI is best used as a reasoning partner rather than as an unquestioned authority.

Use AI for	Example
Model comparison	Compare Logistic Regression, Random Forest, and XGBoost for churn data
Hyperparameter discussion	Explain how max_depth affects overfitting
Feature analysis	Suggest derived features from dates and transaction history
Experiment planning	Design a baseline -> feature engineering -> tuning sequence
Documentation	Summarize experiment findings in clear language

8.1 Validation Checklist for AI Recommendations

- Does the recommendation match the target type?
- Does it consider class imbalance and metric choice?
- Does preprocessing cause leakage?
- Are suggested features actually available before prediction time?
- Do experiments confirm the recommendation?
- Can the result be explained to the intended audience?

9. Practical Setup

The practical exercises at the end of this material use Python and common machine learning libraries. They are designed to run with dummy data so students can practice without downloading an external dataset.

```
# Recommended installation
pip install pandas numpy scikit-learn matplotlib

# Optional for the XGBoost exercise
pip install xgboost
```

- Use Python 3.10+ if possible.
- Run each exercise in a fresh notebook cell.
- Read the printed metrics and compare them with the expected interpretation.
- Change one input or hyperparameter at a time to understand its effect.

10. Practical Questions and Answers (Using Dummy Data)

The following exercises are arranged from basic to intermediate level. Every answer creates its own dummy dataset, trains a model, and demonstrates the expected workflow. Exact metric values can vary slightly across library versions, but the interpretation should remain the same.

Practical 1: Simple Linear Regression

Question: Create a dummy dataset with years of experience and salary. Train a Linear Regression model, predict salary for 6.5 years of experience, and evaluate using MAE and R2.

Answer:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

data = pd.DataFrame({
    "experience": [1,2,3,4,5,6,7,8,9,10,11,12],
    "salary": [24000,29000,35000,42000,50000,57000,65000,73000,81000,89000,96000,105000]
})

X = data[["experience"]]
y = data["salary"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
new_salary = model.predict(pd.DataFrame({"experience": [6.5]}))[0]

print("Predicted salary for 6.5 years:", round(new_salary, 2))
print("MAE:", round(mean_absolute_error(y_test, y_pred), 2))
print("R2:", round(r2_score(y_test, y_pred), 3))
```

Explanation: The model learns an approximately linear relationship between experience and salary. MAE gives the average error in salary units. R2 close to 1 indicates that the linear relationship explains most of the variation in the dummy target.

Practical 2: Multiple Linear Regression

Question: Predict house price using area, bedrooms, and age. Display the learned coefficients and predict the price of a new house.

Answer:

```

import pandas as pd
from sklearn.linear_model import LinearRegression

data = pd.DataFrame({
    "area": [800,1000,1200,1400,1600,1800,2000,2200,2400,2600],
    "bedrooms": [2,2,2,3,3,3,4,4,4,5],
    "age": [15,12,10,9,8,7,6,5,4,3],
    "price": [32,38,44,53,60,67,78,86,94,108] # price in lakh
})

X = data[["area", "bedrooms", "age"]]
y = data["price"]

model = LinearRegression()
model.fit(X, y)

print("Coefficients:", dict(zip(X.columns, model.coef_.round(4))))
print("Intercept:", round(model.intercept_, 3))

new_house = pd.DataFrame({"area": [1750], "bedrooms": [3], "age": [5]})
print("Predicted price (lakh):", round(model.predict(new_house)[0], 2))

```

Explanation: The model uses three features at the same time. Coefficients show how the prediction changes with each feature when the other features are held constant. Because this dataset is small and synthetic, coefficient interpretation is only for practice.

Practical 3: Logistic Regression Classification

Question: Use age and monthly income to predict whether a customer buys a product. Print accuracy, confusion matrix, and predicted probability for a new customer.

Answer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.DataFrame({
    "age": [19,22,24,27,30,32,35,38,40,43,46,50,52,55,58,60],
    "income": [20,22,25,28,32,35,40,45,48,55,60,65,72,78,82,90],
    "purchased": [0,0,0,0,0,0,1,0,1,1,1,1,1,1,1,1]
})

X = data[["age", "income"]]
y = data["purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))

new_customer = pd.DataFrame({"age": [37], "income": [47]})
print("Purchase probability:", model.predict_proba(new_customer)[0,1])

```

Explanation: Logistic Regression estimates the probability of the positive class. The default decision threshold is generally 0.5. The confusion matrix shows which types of classification errors occurred.

Practical 4: KNN with Feature Scaling

Question: Build a KNN classifier on age and income. Use StandardScaler in a Pipeline and compare K=3 with K=5.

Answer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    "age": [18,21,23,25,28,30,33,35,38,40,42,45,48,50,53,56,58,61,64,67],
    "income": [18,20,24,26,30,33,36,40,43,47,50,55,58,62,67,72,76,82,88,95],
    "class": [0,0,0,0,0,0,0,1,0,1,1,1,1,1,1,1,1,1,1,1]
})

X = data[["age", "income"]]
y = data["class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

for k in [3,5]:
    model = Pipeline([
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("K =", k, "Accuracy =", round(accuracy_score(y_test, pred), 3))

```

Explanation: Scaling is important because KNN uses distances. Without scaling, a feature with a larger numeric range can dominate distance calculations. Comparing K values illustrates that hyperparameters affect the decision boundary.

Practical 5: Naive Bayes Text Classification

Question: Create a tiny text dataset and classify messages as spam or not spam using CountVectorizer and Multinomial Naive Bayes.

Answer:

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

texts = [
    "win money now",
    "free prize offer",
    "claim your reward",
    "meeting at office tomorrow",
    "project report attached",
    "team lunch at one",
    "limited offer win prize",
    "please review the document"
]
labels = [1,1,1,0,0,0,1,0] # 1=spam, 0=not spam

model = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("nb", MultinomialNB())
])
model.fit(texts, labels)

new_messages = ["free money prize", "project meeting tomorrow"]
print(model.predict(new_messages))
print(model.predict_proba(new_messages))

```

Explanation: CountVectorizer converts words to numeric counts. Multinomial Naive Bayes learns how strongly words are associated with each class. The first message should receive a higher spam probability than the second.

Practical 6: Decision Tree and Tree Rules

Question: Train a Decision Tree classifier for loan approval and print the learned text rules.

Answer:

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

data = pd.DataFrame({
    "income": [25,30,35,40,45,50,55,60,65,70,80,90],
    "credit_score": [520,560,580,600,620,650,670,690,710,730,760,790],
    "debt_ratio": [0.55,0.50,0.48,0.45,0.42,0.40,0.38,0.35,0.32,0.28,0.25,0.20],
    "approved": [0,0,0,0,0,1,1,1,1,1,1,1]
})

X = data[["income", "credit_score", "debt_ratio"]]
y = data["approved"]

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

print(export_text(model, feature_names=list(X.columns)))

```

Explanation: The printed rules show which features and thresholds the tree used. max_depth limits complexity and makes the tree easier to interpret while reducing overfitting risk.

Practical 7: Random Forest and Feature Importance

Question: Train a Random Forest classifier and rank the features by importance.

Answer:

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

data = pd.DataFrame({
    "income": [25,30,35,40,45,50,55,60,65,70,80,90,95,100,110,120],
    "credit_score": [520,550,570,590,610,630,650,670,690,710,730,750,770,790,810,830],
    "debt_ratio": [0.60,0.55,0.52,0.48,0.45,0.42,0.39,0.36,0.33,0.30,0.28,0.26,0.23,0.21,0.18,0.15],
    "age": [22,24,26,28,30,32,34,36,38,40,42,44,46,48,50,52],
    "approved": [0,0,0,0,0,1,1,1,1,1,1,1,1,1,1,1]
})

X = data.drop(columns="approved")
y = data["approved"]

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X, y)

importance = pd.Series(model.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False))
```

Explanation: Random Forest combines many trees, so its prediction is usually more stable than a single tree. Feature importance helps identify which variables the forest used most strongly, but importance is not proof of cause and effect.

Practical 8: Compare Multiple Classification Algorithms

Question: Train Logistic Regression, KNN, Decision Tree, and Random Forest on the same dummy dataset. Compare accuracy and F1 score.

Answer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

data = pd.DataFrame({
    "age": [20,22,24,26,28,30,32,34,36,38,40,42,44,46,48,50,52,54,56,58,60,62,64,66],
    "income": [20,22,24,27,30,32,35,38,40,43,46,50,53,56,60,64,68,72,76,80,85,90,96,102],
    "visits": [1,1,2,1,2,2,3,2,3,4,3,5,4,5,6,5,7,6,7,8,8,9,10,11],
    "bought": [0,0,0,0,0,0,0,0,1,0,1,1,1,1,1,1,1,1,1,1,1,1,1]
})

X = data[["age", "income", "visits"]]
y = data["bought"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

models = {
    "Logistic": Pipeline([("scale", StandardScaler()), ("model", LogisticRegression())]),
    "KNN": Pipeline([("scale", StandardScaler()), ("model", KNeighborsClassifier(n_neighbors=3))]),
    "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=150, random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name,
          "Accuracy=", round(accuracy_score(y_test, pred), 3),
          "F1=", round(f1_score(y_test, pred), 3))

```

Explanation: The exercise compares models under the same split. This is a better practice than choosing an algorithm based only on popularity. In real projects, use cross-validation and additional metrics before making a final decision.

Practical 9: Regression Model Comparison

Question: Compare Linear Regression and Random Forest Regression using MAE.

Answer:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

data = pd.DataFrame({
    "area": [700,850,950,1100,1250,1400,1550,1700,1850,2000,2200,2400,2600,2800,3000],
    "bedrooms": [1,2,2,2,3,3,3,3,4,4,4,4,5,5,5],
    "age": [20,18,17,15,14,12,10,9,8,7,6,5,4,3,2],
    "price": [28,33,36,42,50,55,61,67,74,81,90,99,110,121,134]
})

X = data[["area", "bedrooms", "age"]]
y = data["price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.27, random_state=42
)

models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=300, random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "MAE=", round(mean_absolute_error(y_test, pred), 3))

```

Explanation: MAE is directly interpretable in the same unit as the price target. The dummy data is nearly linear, so Linear Regression may perform very well. A Random Forest is useful when relationships are more nonlinear or interactive.

Practical 10: XGBoost Classification (Optional)

Question: Train a basic XGBoost classifier on a dummy churn dataset and print accuracy plus feature importance.

Answer:

```

# Install first if needed: pip install xgboost
import pandas as pd
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    "tenure": [1,2,3,4,5,6,8,10,12,14,16,18,20,22,24,26,30,32,36,40],
    "monthly_spend": [80,75,90,70,85,65,60,58,55,52,50,48,46,44,42,40,38,36,34,32],
    "complaints": [5,4,5,4,3,4,3,3,2,2,2,1,1,1,1,0,1,0,1,0],
    "churn": [1,1,1,1,1,1,1,0,1,0,0,0,0,0,0,0,0,0,0,0]
})

X = data.drop(columns="churn")
y = data["churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

model = XGBClassifier(
    n_estimators=80,
    max_depth=3,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    eval_metric="logloss",
    random_state=42
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print(pd.Series(model.feature_importances_, index=X.columns).sort_values(ascending=False))

```

Explanation: XGBoost builds trees sequentially. The parameters control how many trees are built, how complex each tree is, and how much each tree contributes. The dummy dataset is intentionally small; real XGBoost tuning should use cross-validation on a larger dataset.

Practical 11: Detect Overfitting with Train vs Test Accuracy

Question: Create a noisy classification dataset and compare an unrestricted Decision Tree with a shallow tree.

Answer:

```

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=300, n_features=10, n_informative=4,
    n_redundant=2, flip_y=0.12, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

for depth in [None, 3]:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))
    print("max_depth=", depth,
          "train=", round(train_acc,3),
          "test=", round(test_acc,3))

```

Explanation: The unrestricted tree often gets extremely high training accuracy but lower test accuracy. This gap is evidence of overfitting. A shallow tree may sacrifice a little training accuracy while improving generalization.

Practical 12: Build a Reusable Prediction Pipeline

Question: Build a pipeline that scales features and trains Logistic Regression. Use it to predict a new record and save the model with joblib.

Answer:

```

import pandas as pd
import joblib
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = pd.DataFrame({
    "age": [21,25,28,32,35,38,42,45,48,52,56,60],
    "income": [22,28,32,36,41,46,52,58,64,72,80,90],
    "visits": [1,1,2,2,3,3,4,5,5,6,7,8],
    "purchased": [0,0,0,0,0,1,0,1,1,1,1,1]
})

X = data[["age", "income", "visits"]]
y = data["purchased"]

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])
model.fit(X, y)

new_customer = pd.DataFrame({"age": [40], "income": [50], "visits": [4]})
print("Prediction:", model.predict(new_customer)[0])
print("Probability:", model.predict_proba(new_customer)[0,1])

joblib.dump(model, "purchase_model.joblib")
loaded_model = joblib.load("purchase_model.joblib")
print("Loaded model prediction:", loaded_model.predict(new_customer)[0])

```

Explanation: A Pipeline ensures the same preprocessing is applied during training and prediction. Saving the complete pipeline helps prevent production mistakes such as applying a different scaler or forgetting preprocessing.

11. Theory and Viva Review Questions

11. What is supervised learning?
12. What is labeled data?
13. Differentiate feature, target, and label.
14. How do regression and classification differ?
15. Why is train-test splitting necessary?
16. What does Linear Regression learn?
17. When should Multiple Linear Regression be used?
18. Why is Logistic Regression a classification algorithm?
19. Why should KNN features usually be scaled?
20. What is the naive assumption in Naive Bayes?
21. Explain root node, branch, and leaf in a Decision Tree.
22. How does Random Forest differ from a single Decision Tree?
23. Differentiate bagging and boosting.
24. What is XGBoost?
25. What is overfitting?
26. What is underfitting?
27. Why can accuracy be misleading?
28. Differentiate precision and recall.
29. What is feature importance?
30. Why does feature importance not prove causation?
31. What is experiment tracking?
32. What is data leakage?
33. What is the purpose of cross-validation?
34. How can AI assist model selection?
35. Why must AI-generated recommendations be validated?

12. Quick Reference Summary

Algorithm	Type	Strength	Important caution
Linear Regression	Regression	Simple and interpretable	Limited for nonlinear relationships
Logistic Regression	Classification	Fast, interpretable baseline	Linear decision boundary unless features are engineered
KNN	Both	Simple, local pattern learning	Needs scaling; slow prediction on large data
Naive Bayes	Classification	Fast, effective for text	Strong independence assumption
Decision Tree	Both	Interpretable nonlinear rules	Can overfit easily
Random Forest	Both	Strong general-purpose tree ensemble	Less interpretable than one tree
XGBoost	Both	High performance on tabular data	Needs careful tuning and validation

Task	Recommended first question
Choose regression/classification	What is the target type?
Evaluate regression	How large are prediction errors in target units?
Evaluate classification	Which errors matter most - false positives or false negatives?
Choose model	What metric and operational constraints matter?
Interpret model	Which features influence predictions and in what direction?
Deploy model	Is preprocessing packaged consistently with the model?

End of Module 11 Study Material